

Albertitos · plan de arquitectura y decisiones

HackSpain 2026 · reto Maisa "500 Sombras de Alberto" · equipo: Javier Saguar, Miguel Vera, Alejandro Cuevas, Alfonso Jimena, Mónica Fernández

1. Arquitectura

El problema y para quién

Alberto, administrativo de cuentas a pagar del Banco Miralmar, recibe facturas PDF (algunas escaneadas), un Excel de proveedores y pedidos con su norma de pagos, y un ERP de 2009 con los asientos contables. Tiene que decidir para cada factura PAGAR, NO_PAGAR o ESCALAR, sin pagar dos veces y sin pagar lo que un humano debe ver.

Componentes

(rellenar: diagrama de cajas; una frase por componente; enlazar a `src/albertitos/<modulo>/CLAUDE.md`)

- **Motor por lotes (CLI `albertitos`):** ingest → extract → decide → package.
- **Fuentes:** cliente ERP 2009 (snapshot versionado), loader del Excel (maestro versionado por contenido).
- **Extracción:** PyMuPDF → texto o imagen → LLM con esquema cerrado y caché → validadores deterministas.
- **Norma como código:** `norma_v3.py` (+ `norma_v4.py` el sábado), funciones puras con motivo y evidencia.
- **Estado:** SQLite WAL con `ficheros`, `hechos`, `snapshots`, `decisiones` (historial), `eventos`, `cache_llm`.
- **Consola:** Streamlit de sólo lectura: cola de escalados, traza de una decisión, panel operativo.

Flujo de datos y estado

Etap	Entra	Sale (tabla)	Evento
ingest	PDF	<code>ficheros</code> (sha256, nombre NFC, páginas, ¿texto?)	<code>ingest</code> sólo si es nuevo o cambió
extract	PDF → plantilla, o texto/imagen → LLM	<code>hechos</code> (<code>InvoiceFacts</code> + <code>hechos_hash</code>), <code>cache_llm</code>	<code>extract</code> ok / pendiente / retry con tokens, coste y error
validate	todos los hechos	<code>aviso duplicado_sospechoso</code> (se pone y se quita)	<code>validate</code> con los otros PDF del grupo
enrich	bridge ERP 2009 y Excel	<code>snapshots</code> (ERP v1/v2, maestro por hash)	<code>enrich</code> por consulta (ORA-00600, 429, token)
decide	hechos + maestro + ERP + <code>fecha_corte</code>	<code>decisiones</code> (historial; <code>vigente</code>)	<code>decide</code> ok (resultado, reglas KO, por qué) o skip (pendiente o sin impacto)
emit	decisiones vigentes	<code>dist/entrega/*.jsonl</code> , validado, todo o nada	<code>emit</code> por intento y lote; por fichero, si cambia lo entregado

Reparto entre agentes, modelos y personas

- El **modelo** extrae campos (y lee las 29 escaneadas). No decide.
- El **código** (norma versionada) decide, con motivo y evidencia por regla.

- La **persona** (Alberto) resuelve la cola de ESCALAR con la evidencia delante.
- Los **agentes de código** (Claude Code) construyeron el sistema con contratos congelados y hooks; no forman parte del runtime.

Observabilidad y recuperación

- **Traza:** `albertitos trace <file_id>` y la vista Traza de la consola enseñan hechos (método y tokens), versión del maestro, asiento del ERP (con sus reintentos), las 6 reglas con evidencia y lo entregado. Los eventos de estado sólo se escriben cuando el estado cambia, así que repetir no ensucia la traza.
- **Si el LLM cae:** la factura queda PENDIENTE (sin hechos no hay decisión, y sin decisión no hay PAGAR) y `package` se niega a entregar. La entrega anterior sigue intacta. Al volver el LLM, el mismo `run` reanuda. `make demo-caos` lo enseña de punta a punta en 21,9 s.
- **Sin duplicados al reanudar:** la identidad es el sha256 y la caché del LLM también va por sha256. Dos pasadas dan el mismo JSONL byte a byte.
- **Cambios de datos o norma:** `reprocess --impacted` recalcula sólo lo que el cambio toca y dice por qué (ADR-0006).

Escala, evolución y coste

Medido en un portátil i5-1235U con 7,7 GB y Windows 11, con 4 hilos (`docs/benchmark.md`): - **Pasada completa en frío:** 500 facturas en 204 s (2,5 ficheros/s), APTO. - **Con la caché del LLM llena:** 15 s (33 ficheros/s) y 0 tokens. - **Reprocesar un cambio de ERP:** 2 de 500 facturas en 0,04 s.

468 de 500 salen por plantilla (3 ms). El LLM lee 32: 29 escaneadas (p50 17 s) y 3 de texto (3,3 s). **Coste marginal: 0 € por factura** (modelos abiertos en suscripción plana). El coste real es el fijo, 399 €/mes (0,04 € por factura con 10.000 al mes).

El cuello de botella es la visión, que satura con 4 hilos (0,22 ficheros/s). Escalar 10× no exige pagar más LLM, sino que menos facturas lleguen a él (más plantillas). Un formato nuevo (email, Excel) es un conector de `extract/` que produce el mismo `InvoiceFacts`; la norma no cambia.

2. ADRs / trade-offs

(2 a 5. Resumen de 5 líneas de cada uno; el detalle está en `docs/adr/`)

ADR-0001 · El LLM extrae; la norma decide

Hay 12 PDFs que intentan dictar la decisión ("escalar", "ignorar NIF", "pagar el total impreso") y la validación es binaria. Por eso el LLM sólo rellena `InvoiceFacts`, un esquema cerrado sin campo de decisión; las instrucciones se guardan como aviso y evidencia. Decide `rules/norma_v3.py`: seis funciones puras con motivo y evidencia, versionadas (la v4 es un módulo nuevo). 468 de 500 facturas salen por plantilla determinista y el LLM lee 32; coste marginal 0 EUR. Riesgo aceptado: escalados de más a cambio de cero PAGAR inducidos por el documento. Si el LLM cae, PENDIENTE, nunca PAGAR.

ADR-0006 · Reprocesar sólo lo que el cambio toca

Una decisión depende de sus hechos, de la norma, de la fecha de corte y del pedido y el NIF de la factura en el maestro y el ERP. `reprocess --impacted` recalcula sólo las decisiones cuyo pedido o NIF toca el diff de maestro o ERP, y dice por qué. Las demás conservan la versión con la que se decidieron y dejan un evento "sin impacto": no se reescribe el historial. Ensayo ERP v1→v2-sim: **2 de 500 recalculadas, cambian 2**, en 0,04 s (antes: 510 de 510). Vuelta a v1: estado idéntico al original. Riesgo aceptado: depende de que la norma lea sólo esas claves. Un test lo comprueba en cada norma registrada, y `--todo` es la salida.

ADR-0007 · Una CLI que escribe, una consola que sólo lee, traza en la entrega

Lo que se entrega son ficheros validados de forma exacta, así que el único que escribe en la BD es la CLI `albertitos`. La consola Streamlit sólo lee: cola de escalados, traza y panel, con una única acción (reprocesar, que llama a la CLI). Cada línea entregada lleva `motivo`, `norma_version` y `regla` (permitidos por el verificador); `--sin-traza` los quita. Toda entrega pasa por un validador que replica el oficial y se escribe todo o nada: una inválida no pisa la anterior. Se descartan una API web con SPA (red e infraestructura) y una consola que opere (dos escritores, correcciones sin regla).

ADR-0008 · SQLite como única fuente de verdad

Seis tablas en una SQLite WAL: ficheros (sha256), hechos, snapshots versionados, decisiones con historial, eventos y caché del LLM. La traza de una factura es una consulta; el reprocesado usa el historial; la idempotencia sale de las claves (sha256). Postgres (infraestructura en cinco portátiles), JSONL por etapa (sin atomicidad ni cruces) y ORM (coste sin beneficio): descartados. Riesgo aceptado: un solo escritor a la vez; ya nos mordió (`database is locked` con 4 hilos) y se resolvió con commit por fichero. 3,3 MB para 500 facturas y 1.000 decisiones; reprocesar 500 decisiones, 0,04 s.

ADR-000N · ...